

ELK Stack + Sysmon + Wazuh

Full SOC Home Lab on GCP

Complete Step-by-Step Guide — From Zero to Live Detections

SIEM Engineering

Detection Engineering

Threat Hunting

Incident Response

Author: Samson Ejim
Cybersecurity Enthusiast | Nigeria

Google Cloud Platform | Ubuntu 22.04 LTS | ELK 8.19 | Sysmon 15 | Wazuh 4.7.5

This guide documents the complete build of a production-grade SOC home lab combining the Elastic Stack, Microsoft Sysmon, and Wazuh SIEM on Google Cloud Platform. You will deploy the full ingestion pipeline, write custom KQL detection rules, simulate real adversary techniques using Atomic Red Team, and package the entire project as a professional cybersecurity portfolio piece.

PREREQUISITE	Completed Wazuh + VirusTotal on GCP (Project 1)
PREREQUISITE	Completed Suricata IDS + Wazuh on GCP (Project 2)
PREREQUISITE	Completed Velociraptor DFIR + Wazuh on GCP (Project 3)
LEVEL	Intermediate SOC Analyst
DURATION	6–10 hours build + ongoing detection practice

GITHUB	github.com/samsonejim/elk-sysmon-soc-lab
--------	---

Table of Contents

1. Architecture Overview

- Lab topology
- Component roles
- Data flow

2. GCP Infrastructure Setup

- VM creation via GUI
- Firewall rules
- Static IP setup

3. Elasticsearch Installation

- Install & configure
- Security setup
- Cluster health check

4. Logstash Pipelines

- Winlogbeat pipeline (port 5044)
- Wazuh pipeline (port 5045)
- SSL certificate config

5. Kibana Installation

- Install & configure
- Enrollment token method
- Index patterns

6. Wazuh Integration with ELK

- Python alert shipper
- Wazuh field parsing fix
- Verify data in Kibana

7. Sysmon Deployment on Windows

- Install via PowerShell
- SwiftOnSecurity config
- Key Event IDs reference

8. Winlogbeat Configuration

- Install & configure
- Ship to Logstash
- Verify in Kibana

9. KQL Detection Rules

- 10 custom rules
- MITRE ATT&CK; mappings
- Elastic prebuilt rules

10. Kibana SOC Dashboards

- SOC Overview Dashboard
- Threat Alerts Dashboard
- Threat Hunting Dashboard

11. Attack Simulations

- Atomic Red Team setup
- 8 attack scenarios
- Alert verification

12. Portfolio Documentation

- GitHub repository
- Screenshot guide
- LinkedIn post template

13. Key Learnings & Troubleshooting

- Common errors & fixes
- Version mismatch solutions
- Firewall gotchas

1. Architecture Overview

This lab deploys three GCP virtual machines on the same VPC network, plus a local Windows VM in VirtualBox. The architecture mirrors a real enterprise SOC environment.

VM / Host	Role	OS	vCPU	RAM	Disk
elk-server	Elasticsearch + Logstash + Kibana	Ubuntu 22.04	4	16 GB	100 GB SSD
wazuh-server	Wazuh Manager + Indexer + Dashboard	Ubuntu 22.04	4	16 GB	50 GB SSD
ubuntu-endpoint	Suricata IDS + Wazuh Agent	Ubuntu 22.04	2	4 GB	20 GB SSD
windows-endpoint	Sysmon + Winlogbeat + Wazuh Agent	Win Server 2022	2	4 GB	60 GB VDI

Data Flow

Windows VM (Sysmon events) → **Winlogbeat** (port 5044) → **Logstash** (parses & enriches) → **Elasticsearch** (indexes) → **Kibana** (detects & visualises)

Wazuh Manager (alerts.json) → **Python Shipper** (TCP port 5045) → **Logstash** → **Elasticsearch** (wazuh-alerts-* index) → **Kibana**

i Why ELK + Wazuh together?

Wazuh excels at host-based detection (FIM, rootkit detection, compliance checks). ELK adds powerful log analytics, custom KQL detections, and rich dashboards. Together they replicate the stack used at major financial institutions and MSSPs globally.

! GCP Free Trial limitation

Windows Server VMs cannot be created on GCP Free Trial. The windows-endpoint VM must run locally in VirtualBox. Winlogbeat ships logs from your local PC to GCP Logstash over the internet — this requires opening port 5044 to 0.0.0.0/0 in your GCP firewall.

2. GCP Infrastructure Setup

All VM creation is done via the GCP Console GUI. Navigate to Compute Engine → VM Instances → Create Instance.

2.1 ELK Server VM Settings

Field	Value
Name	elk-server
Region	us-central1 (Iowa)
Zone	us-central1-a
Machine family	General purpose
Series	E2
Machine type	e2-standard-4 (4 vCPU, 16 GB RAM)
Boot disk OS	Ubuntu 22.04 LTS
Boot disk type	SSD persistent disk
Boot disk size	100 GB
Network tags	elk-server
Network	default

2.2 Firewall Rules

Rule Name	Target Tag	Port	Source	Purpose
allow-kibana	elk-server	5601/TCP	0.0.0.0/0	Kibana web UI
allow-logstash-beats	elk-server	5044/TCP	0.0.0.0/0	Winlogbeat (local VM over internet)
allow-logstash-wazuh	elk-server	5045/TCP	0.0.0.0/0	Wazuh Python shipper
allow-wazuh-api	wazuh-manager	55000/TCP	10.128.0.0/9	Wazuh API (internal only)

! Security note

Ports 5044 and 5045 are opened to 0.0.0.0/0 only because your local Windows VM ships logs from your home internet connection. In a production environment these would be locked to specific source IPs. Always stop your GCP VMs when not in use.

2.3 Reserve Static External IPs

Go to VPC Network → IP Addresses → Reserve External Static Address. Reserve a static IP for elk-server so your Kibana URL never changes when you stop/start the VM.

3. Elasticsearch Installation

SSH into elk-server and run the following commands in order. Do not skip ahead.

3.1 System update

```
■ elk-server ~$ ssh
$ sudo apt update && sudo apt upgrade -y
```

3.2 Import GPG key and add repository

```
■ elk-server ~$ ssh
$ wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg
$ echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/8.x/apt stable main" | sudo tee /etc/apt/sources.list.d/elasticsearch-8.x.list
```

3.3 Install Elasticsearch

```
■ elk-server ~$ ssh
$ sudo apt update && sudo apt install elasticsearch -y
# IMPORTANT: Copy the auto-generated elastic password from the output!
# It looks like: The generated password for the elastic built-in superuser is: XXXXXX
```

3.4 Enable and start

```
■ elk-server ~$ ssh
$ sudo systemctl daemon-reload
$ sudo systemctl enable elasticsearch
$ sudo systemctl start elasticsearch

# Verify — expect: 'status': 'green' or 'yellow'
$ curl -k -u elastic:YOUR_PASSWORD https://localhost:9200/_cluster/health?pretty
```

v Expected cluster health output

"status" : "green" (single node may show "yellow" — both are fine)

"number_of_nodes" : 1

"tagline" : "You Know, for Search"

! Save your password immediately

The auto-generated elastic password is shown ONCE during installation. Save it to a password manager immediately. If lost, reset with: `sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic`

4. Logstash Pipelines

Logstash acts as the central broker — receiving logs from Winlogbeat and Wazuh, parsing them, and forwarding to Elasticsearch. Two separate pipelines run on different ports.

4.1 Install Logstash

```
■ elk-server ~$ ssh  
sudo apt install logstash -y
```

4.2 Copy Elasticsearch CA certificate

```
■ elk-server ~$ ssh  
sudo cp /etc/elasticsearch/certs/http_ca.crt /etc/logstash/http_ca.crt  
sudo chown logstash:logstash /etc/logstash/http_ca.crt  
sudo chmod 640 /etc/logstash/http_ca.crt
```

4.3 Winlogbeat pipeline — /etc/logstash/conf.d/winlogbeat.conf

```
■ /etc/logstash/conf.d/winlogbeat.conf  
input {  
  beats { port => 5044 host => '0.0.0.0' type => 'winlogbeat' }  
}  
filter {  
  if [type] == 'winlogbeat' {  
    if [winlog][event_id] {  
      mutate { add_field => { 'sysmon_event_id' => '%{[winlog][event_id]}' } }  
    }  
    if [winlog][event_id] == 1 { mutate { add_tag => ['sysmon_process_create'] } }  
    if [winlog][event_id] == 3 { mutate { add_tag => ['sysmon_network_connect'] } }  
    if [winlog][event_id] == 10 { mutate { add_tag => ['sysmon_lsass_access'] } }  
  }  
}  
output {  
  if [type] == 'winlogbeat' {  
    elasticsearch {  
      hosts => ['https://localhost:9200']  
      user => 'elastic' password => 'YOUR_ELASTIC_PASSWORD'  
      ssl_enabled => true  
      ssl_certificate_authorities => ['/etc/logstash/http_ca.crt']  
      index => 'winlogbeat-%{+YYYY.MM.dd}'  
    }  
  }  
}
```

```
}
```

4.4 Wazuh pipeline — /etc/logstash/conf.d/wazuh.conf

```
# /etc/logstash/conf.d/wazuh.conf
input {
  tcp { port => 5045 host => '0.0.0.0' codec => json_lines type => 'wazuh-alerts' }
  beats { port => 5046 host => '0.0.0.0' type => 'wazuh-beats' }
}
filter {
  if [type] == 'wazuh-alerts' {
    mutate { add_field => { 'log_source' => 'wazuh' } }
  }
}
output {
  if [type] == 'wazuh-alerts' or [type] == 'wazuh-beats' {
    elasticsearch {
      hosts => ['https://localhost:9200']
      user => 'elastic' password => 'YOUR_ELASTIC_PASSWORD'
      ssl_enabled => true
      ssl_certificate_authorities => ['/etc/logstash/http_ca.crt']
      index => 'wazuh-alerts-%{+YYYY.MM.dd}'
    }
  }
}
```

4.5 Start Logstash

```
# elk-server ~ bash
sudo systemctl enable logstash && sudo systemctl start logstash
# Verify both ports are listening
sudo tail -10 /var/log/logstash/logstash-plain.log | grep 'Starting server'
# Expected: Starting server on port: 5044 AND Starting server on port: 5045
```

5. Kibana Installation & Configuration

5.1 Install Kibana

```
■ elk-server — bash
sudo apt install kibana -y
```

5.2 Generate enrollment token

```
■ elk-server — bash
sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana
# Copy the long base64 token — you need it in the next step
```

5.3 Configure Kibana

```
■ elk-server — bash
# Write basic config
sudo tee /etc/kibana/kibana.yml > /dev/null << 'EOF'
server.port: 5601
server.host: '0.0.0.0'
server.name: 'soc-lab-kibana'
EOF

# Link to Elasticsearch using enrollment token
sudo /usr/share/kibana/bin/kibana-setup --enrollment-token YOUR_TOKEN_HERE
```

5.4 Add encryption keys (required for Security/SIEM module)

Generate three random 32-character keys:

```
■ elk-server — bash
cat /dev/urandom | tr -dc 'a-zA-Z0-9' | fold -w 32 | head -3
# Copy all three outputs, then add to kibana.yml
sudo tee -a /etc/kibana/kibana.yml > /dev/null << 'EOF'
xpack.security.encryptionKey: 'KEY_1_HERE'
xpack.encryptedSavedObjects.encryptionKey: 'KEY_2_HERE'
xpack.reporting.encryptionKey: 'KEY_3_HERE'
EOF
```

! Encryption keys are required

Without these three keys, the Kibana Security module will show: 'Failed to retrieve detection engine privileges' and you cannot create detection rules. Always add them before accessing Security → Rules.

5.5 Start Kibana and verify

```
■ elk-server ~$ ssh
$ sudo systemctl enable kibana && sudo systemctl start kibana
# Wait 3 minutes then verify
$ curl http://localhost:5601/api/status | python3 -m json.tool | grep level
# Expected: 'level': 'available'

# Access Kibana at: http://YOUR_ELK_EXTERNAL_IP:5601
# Login: elastic / YOUR_ELASTIC_PASSWORD
```

5.6 Create Data Views in Kibana

Index Pattern	Time Field	Contains
winlogbeat-*	@timestamp	Windows event logs + Sysmon telemetry
wazuh-alerts-*	@timestamp	Wazuh detections, FIM, compliance alerts

Navigate to: Stack Management → Data Views → Create data view

6. Wazuh Integration with ELK

The default Wazuh Filebeat 7.10.2 installation has a compatibility issue with Ubuntu 22.04 kernels that causes repeated crashes. The solution is a lightweight Python shipper that reads the Wazuh alerts JSON file and forwards to Logstash via raw TCP.

6.1 Create the Python alert shipper — on wazuh-server

```
■ wazuh-server ~$ bash
sudo tee /usr/local/bin/wazuh-shipper.py > /dev/null << 'EOF'
#!/usr/bin/env python3
import json, socket, time, os

ALERTS_FILE = '/var/ossec/logs/alerts/alerts.json'
LOGSTASH_HOST = '10.128.0.7' # elk-server internal IP
LOGSTASH_PORT = 5045
POSITION_FILE = '/var/lib/wazuh-shipper/position'

def get_position():
    try:
        with open(POSITION_FILE) as f: return int(f.read().strip())
    except: return 0

def save_position(pos):
    os.makedirs(os.path.dirname(POSITION_FILE), exist_ok=True)
    with open(POSITION_FILE, 'w') as f: f.write(str(pos))

def send(data):
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.connect((LOGSTASH_HOST, LOGSTASH_PORT))
        sock.sendall((json.dumps(data) + chr(10)).encode())
        sock.close(); return True
    except Exception as e: print(f'Error: {e}'); return False

def tail_alerts():
    pos = get_position()
    while True:
        try:
            with open(ALERTS_FILE) as f:
                f.seek(pos)
                while True:
```

```

line = f.readline()
if not line: time.sleep(2); break
line = line.strip()
if line:
    try:
        alert = json.loads(line)
        alert['type'] = 'wazuh-alerts'
        if send(alert): pos = f.tell(); save_position(pos)
    except json.JSONDecodeError: pass
except Exception as e: print(f'Error: {e}'); time.sleep(5)

if __name__ == '__main__':
    print('Wazuh shipper starting...'); tail_alerts()
EOF
sudo chmod +x /usr/local/bin/wazuh-shipper.py

```

6.2 Create systemd service

```

# wazuh-server ~ book
sudo tee /etc/systemd/system/wazuh-shipper.service > /dev/null << 'EOF'
[Unit]
Description=Wazuh Alert Shipper
After=network.target
[Service]
Type=simple
ExecStart=/usr/bin/python3 /usr/local/bin/wazuh-shipper.py
Restart=always
RestartSec=5
User=root
[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable wazuh-shipper
sudo systemctl start wazuh-shipper

```

v Verify wazuh data is flowing

On elk-server run: `curl -k -u elastic:PASSWORD https://localhost:9200/_cat/indices?v | grep wazuh`

You should see: `wazuh-alerts-YYYY.MM.DD` with document count > 0

In Kibana Discover, select `wazuh-alerts-*` — you should see 80+ available fields including `rule.level`, `rule.description`, `agent.name`

i Why not Filebeat?

Filebeat 7.10.2 (pinned by Wazuh) has a known segfault on Ubuntu 22.04 kernels 6.x. The Python shipper is more reliable, easier to debug, and produces the same result. This is a documented workaround for this version combination.

7. Sysmon Deployment on Windows

Sysmon (System Monitor) is a Windows system service that logs detailed process, network, and file system activity. It is the gold standard for endpoint telemetry in enterprise SOCs.

7.1 Install Sysmon — Windows VM PowerShell (Administrator)

```
■ Windows VM — PowerShell (Admin)
# Download Sysmon
Invoke-WebRequest -Uri 'https://download.sysinternals.com/files/Sysmon.zip' -OutFile
'C:\Sysmon.zip'
Expand-Archive -Path 'C:\Sysmon.zip' -DestinationPath 'C:\Sysmon'

# Download SwiftOnSecurity config (best practice)
Invoke-WebRequest -Uri 'https://raw.githubusercontent.com/SwiftOnSecurity/sysmon-config/master/
sysmonconfig-export.xml' -OutFile 'C:\Sysmon\sysmonconfig.xml'

# Install with config
C:\Sysmon\Sysmon64.exe -accepteula -i C:\Sysmon\sysmonconfig.xml

# Verify
Get-Service Sysmon64
```

7.2 Key Sysmon Event IDs

Event ID	Name	What it captures	Detection value
1	ProcessCreate	Every process: image, cmdline, parent, hashes	Critical
3	NetworkConnect	All TCP/UDP connections: src/dst IP, port, process	Critical
7	ImageLoad	DLL loads with hashes — detects DLL injection	Critical
8	CreateRemoteThread	Thread injection into other processes	Critical
10	ProcessAccess	LSASS reads — detects credential dumping	Critical
11	FileCreate	File creation events with hashes	Medium
12/13	RegistryEvents	Registry key creates/modifies — persistence	High
22	DNSQuery	All DNS queries with the process that made them	Critical
25	ProcessTampering	Process image changes — process hollowing	Critical

8. Winlogbeat Configuration

Winlogbeat ships Windows event logs and Sysmon telemetry from your local VM to Logstash on GCP.

8.1 Download and install

```
Windows VM - PowerShell (Admin)
Invoke-WebRequest -Uri
'https://artifacts.elastic.co/downloads/beats/winlogbeat/winlogbeat-8.19.0-windows-x86_64.zip'
-OutFile 'C:\winlogbeat.zip'
Expand-Archive -Path 'C:\winlogbeat.zip' -DestinationPath 'C:\'
Rename-Item 'C:\winlogbeat-8.19.0-windows-x86_64' 'C:\winlogbeat'
cd C:\winlogbeat
.\install-service-winlogbeat.ps1
```

8.2 winlogbeat.yml configuration

```
C:\winlogbeat\winlogbeat.yml
winlogbeat.event_logs:
- name: Application
ignore_older: 72h
- name: System
ignore_older: 72h
- name: Security
ignore_older: 72h
- name: Microsoft-Windows-Sysmon/Operational
ignore_older: 72h
- name: Microsoft-Windows-PowerShell/Operational
ignore_older: 72h
- name: Microsoft-Windows-Windows Defender/Operational
ignore_older: 72h

output.logstash:
hosts: ['YOUR_ELK_SERVER_EXTERNAL_IP:5044']

logging.level: info
logging.to_files: true
logging.files:
path: C:\ProgramData\winlogbeat\Logs
name: winlogbeat
keepfiles: 7
```

8.3 Start and verify

■ Windows VM — PowerShell (Admin)

```
# Test config
.\winlogbeat.exe test config -c C:\winlogbeat\winlogbeat.yml

# Start service
Start-Service winlogbeat
Get-Service winlogbeat

# Verify connection — look for 'Connection established'
Get-Content 'C:\ProgramData\winlogbeat\Logs\winlogbeat*.ndjson' -Tail 5
```

v Success indicators

Get-Service winlogbeat shows Status: Running

Log file shows: Connection to backoff(async(tcp://ELK_IP:5044)) established

Kibana Discover with winlogbeat-* shows documents with agent.name: windows-endpoint

! VirtualBox clipboard

Copy/paste between your host PC and Windows VM is disabled by default. Enable it at: Devices → Shared Clipboard → Bidirectional. If it still doesn't work, install VirtualBox Guest Additions via Devices → Insert Guest Additions CD.

9. KQL Detection Rules

10 custom detection rules written from scratch in Kibana's Security SIEM module, each mapped to a MITRE ATT&CK; technique. Navigate to: Security → Rules → Detection rules (SIEM) → Create new rule.

9.1 Rule summary — all 10 custom rules

Rule Name	Query (KQL)	Technique	Severity	Score
LSASS Memory Access	winlog.event_id:10 AND event_data.TargetImage:Tlsapi.dll	T1543.001	Critical	95
PowerShell Encoded Cmd	event_id:1 AND Image:*powershell.exe AND Command:(*EncodedCommand*OR *-enc*)	T1059.004	High	76
Registry Run Key	event_id:13 AND TargetObject:(*CurrentVersion\Run*)	T1577.001	High	73
Scheduled Task	event_id:1 AND Image:*schtasks.exe AND Command:create	T1053.005	High	73
PowerShell Network	event_id:3 AND Image:(*powershell.exe OR *powershell.exe)	T1071.001	High	75
Brute Force Logins	channel:Security AND event_id:4625 AND LogonType:(3 OR 10)	T1484.001	Medium	60
WMI Child Process	event_id:1 AND ParentImage:(*wmic.exe OR *Wmic.exe)	T1043.001	High	73
Psexec Lateral Move	event_id:1 AND (Image:*psexec* OR Image:*PsExec.exe)	T1569.002	High	75
Event Log Cleared	channel:Security AND event_id:1102	T1070.001	Critical	90
DLL Injection	event_id:8 AND TargetImage:*exe AND NOT SourceImage:*System.dll	T1055.001	Critical	85

9.2 Rule creation workflow

For each rule follow these steps:

1. Security → Rules → Detection rules (SIEM) → Create new rule
2. Select rule type: Custom query
3. Set Index patterns: winlogbeat-* (keep all defaults too)
4. Paste the KQL query into the Custom query box
5. Click Continue → fill in Name, Description, Severity, Risk score
6. Add MITRE ATT&CK; tactic, technique, and subtechnique
7. Schedule: runs every 5 minutes, look-back 1 minute
8. Click Create & enable rule

9.3 Install Elastic prebuilt rules

In addition to your 10 custom rules, install all 1,677 Elastic prebuilt rules: Security → Rules → Add Elastic rules → Install all → Install and enable.

i Encryption keys required first

The Security/SIEM module will show an error if the three xpack encryption keys are not set in kibana.yml. See Section 5.4 for the fix. This is the most common setup error in this project.

10. Kibana SOC Dashboards

Three custom dashboards built in Kibana's Lens editor. Navigate to Analytics → Dashboards → Create dashboard.

10.1 SOC Overview Dashboard

Top-level situational awareness. Set auto-refresh to 30 seconds.

Panel Title	Configuration
Windows Event Volume by Event ID	Bar vertical stacked winlogbeat-* X=@timestamp Y=Count Breakdown=winlog.event_id.keyword (Top 10)
Top 10 Processes Launched	Treemap winlogbeat-* KQL: winlog.event_id:1 Group by=winlog.event_data.Image.keyword (Top 10)
Top Source IPs	Datatable winlogbeat-* KQL: winlog.event_id:3 Field=winlog.event_data.SourceIp.keyword
Windows Login Activity	Bar stacked winlogbeat-* KQL: winlog.event_id:(4624 OR 4625) Breakdown=event_id.keyword
Sysmon Network Connections	Datatable winlogbeat-* KQL: winlog.event_id:3 DestinationIp + DestinationPort
Critical Sysmon Events Timeline	Line winlogbeat-* KQL: winlog.event_id:(1 OR 3 OR 8 OR 10 OR 22)

10.2 Threat Alerts and Detections Dashboard

Daily alert triage view. Focus on actionable detections.

Panel Title	Configuration
LSASS Access Attempts	Datatable winlogbeat-* KQL: winlog.event_id:10 AND TargetImage:*lsass.exe
PowerShell Executions	Datatable winlogbeat-* KQL: event_id:1 AND Image:*powershell.exe CommandLine.keyword
Failed Login Attempts Over Time	Bar winlogbeat-* KQL: winlog.event_id:4625 X=@timestamp
New Services Created	Line winlogbeat-* KQL: winlog.event_id:7045 X=@timestamp

10.3 Threat Hunting Dashboard

Deep investigation view for proactive threat hunting sessions.

Panel Title	Configuration
Process Parent-Child Relationships	Datatable winlogbeat-* KQL: event_id:1 ParentImage + Image
DNS Query Analysis	Datatable winlogbeat-* KQL: event_id:22 QueryName.keyword (Top 20)
Registry Modifications	Datatable winlogbeat-* KQL: event_id:(12 OR 13) TargetObject.keyword

11. Attack Simulations with Atomic Red Team

Atomic Red Team is a library of self-contained attack tests mapped to MITRE ATT&CK.; Run these on your Windows VM to verify detections fire correctly.

11.1 Setup

```

■ Windows VM — PowerShell (Admin)
# Disable Windows Defender first (lab environment only)
Set-MpPreference -DisableRealtimeMonitoring $true
Set-MpPreference -DisableBehaviorMonitoring $true
Add-MpPreference -ExclusionPath 'C:\AtomicRedTeam'

# Install Atomic Red Team
IEX (IWR 'https://raw.githubusercontent.com/redcanaryco/atomicredteam/master/install-atomicredteam.ps1' -UseBasicParsing);
Install-AtomicRedTeam -getAtomics -Force

# Import module
Import-Module 'C:\AtomicRedTeam\invoke-atomicredteam\Invoke-AtomicRedTeam.psd1' -Force

# Verify: should show 330+ techniques
(Get-ChildItem C:\AtomicRedTeam\atomics -Directory).Count

```

11.2 Attack scenarios

#	Technique	Command	Expected Detection	Result
1	T1003.001 — LSASS Dump	Invoke-AtomicTest T1003.001 -TestNumbers 1	LSASS Memory Access rule	Detected
2	T1059.001 — Mimikatz	Invoke-AtomicTest T1059.001 -TestNumbers 1	PowerShell Encoded Cmd rule	Detected
3	T1547.001 — Run Key	Invoke-AtomicTest T1547.001 -TestNumbers 1	Registry Run Key rule	Detected
4	T1053.005 — Sched Task	Invoke-AtomicTest T1053.005 -TestNumbers 1	Scheduled Task rule	Detected
5	T1082 — Sys Discovery	Invoke-AtomicTest T1082 -TestNumbers 1	Process Create logged	Logged
6	T1110.001 — Brute Force	Invoke-AtomicTest T1110.001 -TestNumbers 1	Brute Force rule — 5 alerts	Detected
7	T1218.011 — Rundll32	Invoke-AtomicTest T1218.011 -TestNumbers 1	Process Create logged	Logged
8	T1070.001 — Log Clear	wevtutil cl Security	Event Log Cleared — Critical	Detected

11.3 Verify in Kibana

After running attacks, check these locations in Kibana:

- Security → Alerts — set time to Last 4 hours — should show 29 total alerts

- Analytics → Discover → winlogbeat-* — filter: winlog.event_id:10 for LSASS events
- Threat Hunting Dashboard → DNS Queries — should show raw.githubusercontent.com, download.sysinternals.com
- Threat Alerts Dashboard → PowerShell Executions — should show Mimikatz command lines

11.4 Cleanup after simulations

■ Windows VM — PowerShell (Admin)

Remove attack artefacts

```
Invoke-AtomicTest T1547.001 -TestNumbers 1 -Cleanup
```

```
Invoke-AtomicTest T1053.005 -TestNumbers 1 -Cleanup
```

12. Portfolio Documentation

12.1 Screenshot checklist

#	Screenshot	Location in Kibana	Portfolio value
1	Kibana home screen	Home page after login	Shows tool access
2	Winlogbeat data in Discover	Analytics → Discover → winlogbeat-*	Proves data pipeline
3	Wazuh alerts in Discover	Analytics → Discover → wazuh-alerts*	Proves Wazuh integration
4	Security Alerts — 29 alerts	Security → Alerts → Last 24 hours	Most impactful screenshot
5	Detection Rules — 10 custom	Security → Rules → Custom rules	Shows detection engineering
6	SOC Overview Dashboard	Analytics → Dashboards	Shows dashboard skills
7	Threat Hunting Dashboard	Analytics → Dashboards	Shows hunting capability
8	Threat Alerts Dashboard	Analytics → Dashboards	Shows triage workflow
9	One expanded alert	Click any alert in Security → Alerts	Shows MITRE ATT&CK knowledge

12.2 GitHub repository structure

```

■ github.com/samsonjim/elk-sysmon-soc-lab
elk-sysmon-soc-lab/
|- configs/
|  |- logstash/
|  |  |- winlogbeat.conf
|  |  '- wazuh.conf
|  '- winlogbeat.yml
|- detections/
|  '- kql/
|     |- lsass_credential_dumping.kql
|     |- powershell_encoded_command.kql
|     |- persistence_run_key.kql
|     |- scheduled_task_creation.kql
|     |- powershell_network_connection.kql
|     |- brute_force_login.kql
|     |- wmi_child_process.kql
|     |- psexec_lateral_movement.kql
|     |- event_log_cleared.kql
|     '- dll_injection.kql
|- screenshots/ (9 portfolio screenshots)
'- README.md (full project documentation)

```

! Never commit credentials to GitHub

Before uploading any config file replace all passwords and IP addresses with placeholders:

YOUR_ELASTIC_PASSWORD_HERE and YOUR_ELK_SERVER_EXTERNAL_IP. GitHub automatically scans for leaked credentials and will alert you — but prevention is better.

12.3 LinkedIn post template

■ LinkedIn post

Just completed my 4th cybersecurity home lab project!

ELK Stack + Sysmon + Wazuh – Full SOC Home Lab on GCP

What I built:

- Full SIEM pipeline: Sysmon -> Winlogbeat -> Logstash -> Elasticsearch -> Kibana
- 10 custom KQL detection rules mapped to MITRE ATT&CK
- 1,677 Elastic prebuilt rules installed and running
- 3 SOC dashboards: Overview, Threat Alerts, Threat Hunting
- 8 Atomic Red Team attack simulations including Mimikatz
- 29 real alerts: 1 Critical, 23 High, 5 Medium

Full write-up + configs + KQL rules: github.com/samsonejim/elk-sysmon-soc-lab

#SOCAnalyst #Cybersecurity #ELKStack #SIEM #BlueTeam #MITREATTaCK #DetectionEngineering

13. Key Learnings & Troubleshooting

13.1 Lessons learned

- **Version mismatch handling** — Filebeat 7.x cannot ship directly to Elasticsearch 8.x. Route through Logstash which accepts any Beats version. This is a reusable architectural pattern for any mixed-version environment.
- **Port conflict awareness** — Wazuh Indexer already occupies port 9200. Always provision a dedicated VM for Elasticsearch rather than reusing existing Wazuh VMs.
- **Encryption keys are mandatory** — Kibana Security module requires three xpack encryption keys before detection rules will load. This is easy to overlook and time-consuming to debug.
- **Filebeat 7.10.2 crash on Ubuntu 22.04** — The Wazuh-pinned Filebeat version has a known segfault on newer kernels. The Python TCP shipper is a clean workaround that also teaches you low-level log pipeline concepts.
- **Firewall scope for hybrid setups** — When a local VM needs to reach GCP over the internet, GCP firewall rules must open to 0.0.0.0/0, not just internal CIDR ranges.
- **VirtualBox clipboard** — Bidirectional clipboard must be explicitly enabled under Devices menu. Not on by default and easy to forget.
- **Windows Defender blocks attack tools** — Always disable Defender before running Atomic Red Team. Use exclusion paths rather than disabling globally when possible.

13.2 Common errors and fixes

Error	Cause	Fix
Failed to retrieve detection engine privileges	Missing xpack encryption keys in kibana.yml	Add three 32-char random keys to kibana.yml and restart Kibana
Filebeat service failed (exit-code)	Corrupted registry or meta.json on wazuh-server	sudo systemctl stop filebeat && sudo rm -rf /var/lib/filebeat && sudo systemctl start filebeat
wazuh-alerts-* has only 1 field	Logstash using Beats input for TCP data	Switch wazuh Logstash pipeline to tcp input with codec => json_lines
Winlogbeat error: Config OK but no data	External IP changed after VM restart	Update winlogbeat.yml with new elk-server external IP or reserve a static IP
KQL query validation error in rule	Special characters or wrong field names	Test KQL in Discover first, then copy to rule editor
curl: SSL routines: wrong version number	Kibana serving HTTP but curl using HTTPS	Use http:// not https:// for Kibana curl tests

13.3 Startup checklist for each session

1. Start all GCP VMs: elk-server, wazuh-server, ubuntu-endpoint
2. Start Windows VM in VirtualBox (use Save State between sessions)
3. Wait 3 minutes for all services to boot
4. Run health check on elk-server: `sudo systemctl status elasticsearch kibana logstash`
5. Run health check on wazuh-server: `sudo systemctl status wazuh-manager wazuh-shipper`

6. Run health check on Windows VM: Get-Service winlogbeat; Get-Service Sysmon64
7. If Filebeat/shipper is down: `sudo rm -rf /var/lib/filebeat/registry && sudo systemctl start wazuh-shipper`
8. Open Kibana at `http://ELK_EXTERNAL_IP:5601` and confirm data is flowing

13.4 Skills demonstrated in this project

Skill	Evidence in this project
SIEM engineering	Deployed ELK 8.19 from scratch on GCP, configured Elasticsearch security, Logstash pipelines
Endpoint telemetry	Sysmon with SwiftOnSecurity config capturing 15+ event types including process create and network connections
Log pipeline engineering	Beats → Logstash → Elasticsearch pipeline with SSL, custom field extraction and tagging
KQL detection writing	10 custom rules from scratch, all with MITRE ATT&CK mappings, severity scoring and schedules
Threat hunting	3 dashboards built with DNS analysis, process trees, registry monitoring
Adversary simulation	8 Atomic Red Team tests including Mimikatz credential dumping
Incident response	Alert triage in Kibana Security, timeline correlation, alert investigation
Cloud infrastructure	3 GCP VMs, firewall rules, static IPs, VPC networking
Documentation	Professional GitHub repo, README, KQL library, portfolio screenshots

v Project complete

You have built a production-grade SOC home lab that demonstrates skills directly applicable to SOC Tier 1/2 and Detection Engineering roles.

GitHub: github.com/samsonejim/elk-sysmon-soc-lab

Next project on the roadmap: Splunk + Atomic Red Team Detection Lab (Phase 2)